UE4.27.2 - Epic source-only VerifyImport trace

> **Authority:** Original Epic C++ only; no UnrealDB implementations or test outcomes.
>
> **Epic revision:** `d94b38ae3446da52224bedd2568c078f828b4039`
> **Source:** `L:\Source\Engine\UE4\UE 4.27.2\Engine\Source\Runtime\CoreUObject\Private\UObject\LinkerLoad.cpp`
> **Source SHA-256:** `39DB1F3E1A48D829E65991259EAEFB955680CCF6165D5E31B9DB862B02560569`
>
> **Scope:** VerifyImport, VerifyImportInner and LoadImportPackage lambda only. **Not a complete package-loading path or full UE4.27.2 file-format reference.**

FLinkerLoad::VerifyImport

Wrapper, retry through ObjectRedirector, payload/class checks, failure policy

Step	Epic source	Exact C++ statement	Source-derived operation / next step
W01	LinkerLoad.cpp:2792	`FLinkerLoad::EVerifyResult FLinkerLoad::VerifyImport(int32 ImportIndex)`	Enter VerifyImport with an ImportMap index. **Next:** W02
W02	LinkerLoad.cpp:2803	`bool bCrashOnFail = VerifyImportInner(ImportIndex, WarningAppend);`	Run the first VerifyImportInner attempt; its boolean controls later failure response. **Next:** W03
W03	LinkerLoad.cpp:2804	`if (FPlatformProperties::HasEditorOnlyData() == false)`	Disable fatal-on-failure behavior on platforms without editor-only data. **Next:** W04
W04	LinkerLoad.cpp:2813	`if (Import.SourceLinker && Import.SourceIndex == INDEX_NONE && Import.XObject == NULL && !Import.OuterIndex.IsNull() && Import.ObjectName != NAME_ObjectRedirector)`	Check whether a source linker exists but the object was not found, with a valid non-redirector import outer. **Next:** W05 yes, W14 no

Step	Epic source	Exact C++ statement	Source-derived operation / next step
W05	LinkerLoad.cpp:2819	`Import.ClassName = NAME_ObjectRedirector;`	Temporarily substitute ObjectRedirector/CoreUObject and retry VerifyImportInner. **Next:** W06
W06	LinkerLoad.cpp:2826	`if (Import.SourceIndex == INDEX_NONE) //-V547`	Check whether a redirector export was found; failure sets VERIFY_Failed. **Next:** W07 found, W11 missing
W07	LinkerLoad.cpp:2838	`UObjectRedirector* Redir = dynamic_cast<UObjectRedirector*>(Import.SourceLinker->CreateExport(Import.SourceIndex));`	Create and cast the redirector export; do not Preload until the cast has succeeded. **Next:** W15
W15	LinkerLoad.cpp:2840	`if (!Redir)`	If CreateExport failed, set VERIFY_Failed; otherwise set RF_NeedLoad and Preload to read DestinationObject. **Next:** W11 null, W08 exists
W08	LinkerLoad.cpp:2856	`if (DestObject == nullptr)`	Reject an absent redirector destination. **Next:** W09 present, W11 null

Step	Epic source	Exact C++ statement	Source-derived operation / next step
W09	LinkerLoad.cpp:2863	`bool bIsValidClass = DestObject->HasAnyFlags(RF_ClassDefaultObject);`	Accept class-default objects or find original class in the destination's superclass chain. **Next:** W10 valid, W11 invalid
W10	LinkerLoad.cpp:2887	`Result = VERIFY_Redirected;`	Bind XObject to destination and mark the import as redirected. **Next:** W11
W11	LinkerLoad.cpp:2901	`Import.ClassName = OriginalImport.ClassName;`	Restore serialized ClassName and ClassPackage after the fallback. **Next:** W12
W12	LinkerLoad.cpp:2905	`if (Result != VERIFY_Failed) //-V547`	If successful update SourceIndex/SourceLinker from XObject; otherwise restore original import. **Next:** W14 success, W13 failure
W13	LinkerLoad.cpp:2918	`if (bCrashOnFail)`	Apply Epic's fatal or warning/error response according to the original VerifyImportInner policy. **Next:** W14

Step	Epic source	Exact C++ statement	Source-derived operation / next step
W14	LinkerLoad.cpp:2978	`return Result;`	Return VERIFY_Success, VERIFY_Failed, or VERIFY_Redirected. **Next:** Return

FLinkerLoad::VerifyImportInner

Preconditions, outer/linker selection, export matching, private checks, runtime fallback

Step	Epic source	Exact C++ statement	Source-derived operation / next step
I01	LinkerLoad.cpp:3068	`bool FLinkerLoad::VerifyImportInner(const int32 ImportIndex, FString& WarningSuffix)`	Enter the private VerifyImportInner implementation. **Next:** I02
I02	LinkerLoad.cpp:3203	`( (Import.SourceLinker && Import.SourceIndex != INDEX_NONE)`	Return early for already resolved imports or NAME_None identities. **Next:** I22 already verified / None, I03 continue
I22	LinkerLoad.cpp:3209	`return false;`	Early return: already verified or not relevant in this context. **Next:** Return
I03	LinkerLoad.cpp:3221	`if (Import.OuterIndex.IsNull() && Import.ClassName!=NAME_Package )`	Reject inappropriate null outer on a non-Package import. **Next:** I23 inappropriate outer, I04 valid outer

Step	Epic source	Exact C++ statement	Source-derived operation / next step
I23	LinkerLoad.cpp:3225	`return false;`	Return after rejecting null outer on a non-Package import. **Next:** Return
I04	LinkerLoad.cpp:3228	`else if (Import.OuterIndex.IsNull())`	For top-level Package import, invoke LoadImportPackage; otherwise use non-null-outer path. **Next:** L01 package, I05 other
I05	LinkerLoad.cpp:3241	`if (Import.HasPackageName())`	For a non-null outer with explicit PackageName, load that assigned package. **Next:** I06
I06	LinkerLoad.cpp:3253	`if (Import.OuterIndex.IsImport())`	If outer is another import, recursively VerifyImport it. **Next:** I07 import outer, I08 export outer
I07	LinkerLoad.cpp:3281	`if (!Import.SourceLinker)`	Use outer import's source linker only when the current import has none. **Next:** I09

Step	Epic source	Exact C++ statement	Source-derived operation / next step
I08	LinkerLoad.cpp:3288	`check(Import.OuterIndex.IsExport());`	Export outer requires an explicit package name and package-linker acquisition. **Next:** I09
I09	LinkerLoad.cpp:3301	`if( Import.SourceLinker )`	When a source linker exists, search its hashed ExportMap; otherwise continue to runtime lookup. **Next:** I10 linker, I16 none
I10	LinkerLoad.cpp:3310	`bool bMatchesWithoutShortening = false;`	First find whether any full class-package match exists; only otherwise allow short package names. **Next:** I11
I11	LinkerLoad.cpp:3340	`for( int32 j=Import.SourceLinker->ExportHash[iHash]; j!=INDEX_NONE; j=Import.SourceLinker->ExportMap[j].HashNext )`	Search matching export names/class identities through Epic's ExportHash chains. **Next:** I12

Step	Epic source	Exact C++ statement	Source-derived operation / next step
I12	LinkerLoad.cpp:3359	`if( Import.OuterIndex.IsImport() )`	Validate export outer against the resolved import outer, including cross-linker import identity. **Next:** I13 candidate valid, I16 no candidate
I13	LinkerLoad.cpp:3403	`auto IsPrivateImportAllowed = [this](int32 InImportIndex)`	Apply editor-only private-import containment predicates. **Next:** I14
I14	LinkerLoad.cpp:3412	`const bool bIsImportPublic = !!(SourceExport.ObjectFlags & RF_Public);`	For private exports apply SafeReplace rules and references from consumer import/export maps. **Next:** I15 accepted, I16 not bound
I15	LinkerLoad.cpp:3468	`Import.SourceIndex = j;`	Store the matching export index in the source linker. **Next:** I16
I16	LinkerLoad.cpp:3477	`if (!Pkg && TmpPkg &&`	Handle in-memory-only/instanced package cases before fallback object search. **Next:** I17

Step	Epic source	Exact C++ statement	Source-derived operation / next step
I17	LinkerLoad.cpp:3500	`if( (Pkg == nullptr) && ((LoadFlags & LOAD_FindIfFail) != 0) )`	When requested, allow the ANY_PACKAGE fallback search. **Next:** I18
I18	LinkerLoad.cpp:3534	`UObject* FindObject = FindImportFast(FindClass, bAnyPackage ? nullptr : FindOuter, Import.ObjectName, bAnyPackage);`	Try native/transient/class-default-object lookup under Epic's conditions. **Next:** I19
I19	LinkerLoad.cpp:3570	`if (!Import.XObject && !SafeReplace)`	Return true only when the wrapper must treat missing import/redirector as fatal. **Next:** I21 fatal policy, I20 otherwise
I21	LinkerLoad.cpp:3572	`return true;`	Ask wrapper to treat unredirected import failure as fatal. **Next:** Return
I20	LinkerLoad.cpp:3581	`return false;`	Otherwise return false from VerifyImportInner. **Next:** Return

VerifyImportInner::LoadImportPackage

Nested lambda invoked for top-level Package and explicit PackageName imports

Step	Epic source	Exact C++ statement	Source-derived operation / next step
L01	LinkerLoad.cpp:3072	`auto LoadImportPackage = [this, ImportIndex](FObjectImport& Import, TOptional<FScopedSlowTask>& SlowTask) -> UPackage*`	Enter the lambda invoked for top-level package imports and explicit PackageName imports. **Next:** L02
L02	LinkerLoad.cpp:3082	`FName PackageToLoad = !Import.HasPackageName() ? Import.ObjectName : Import.GetPackageName();`	Select ObjectName or explicit PackageName; apply instancing remap. **Next:** L03
L03	LinkerLoad.cpp:3095	`Package = FindObjectFast<UPackage>(nullptr, PackageToLoadInto);`	Locate an existing in-memory UPackage; load when absent or not fully loaded. **Next:** L04

Step	Epic source	Exact C++ statement	Source-derived operation / next step
L04	LinkerLoad.cpp:3122	`Package = LoadPackageInternal(Package, *PackageToLoad.ToString(), InternalLoadFlags | LOAD_IsVerifying, this, nullptr, nullptr);`	Load package with LOAD_IsVerifying and controlled inherited flags. **Next:** L05
L05	LinkerLoad.cpp:3124	`#if WITH_IOSTORE_IN_EDITOR`	For cooked IoStore packages in editor, populate imported objects from the loaded package. **Next:** L06
L06	LinkerLoad.cpp:3145	`Package = CreatePackage(*PackageToLoad.ToString());`	Create missing UPackage shell, including native cases where disk loading returned null. **Next:** L07
L07	LinkerLoad.cpp:3150	`if (!Package || Package->HasAnyPackageFlags(PKG_Compiling))`	Reject null/compiling packages and skip verification while gathering dependencies. **Next:** L08 non-compiling, L10 null or compiling

Step	Epic source	Exact C++ statement	Source-derived operation / next step
L10	LinkerLoad.cpp:3152	`return nullptr;`	Return no package when null or PKG_Compiling; no source linker can be acquired in this branch. **Next:** Return
L08	LinkerLoad.cpp:3176	`Import.SourceLinker = !bWasFullyLoaded ? GetPackageLinker(Package, nullptr, InternalLoadFlags, nullptr, nullptr, nullptr, &SerializeContext) : FindExistingLinkerForPackage(Package);`	Acquire an existing or new source linker according to package and cooked/editor state. **Next:** L09
L09	LinkerLoad.cpp:3186	`return Package;`	Return UPackage to the calling VerifyImportInner branch. **Next:** Return

Source-only navigation. Use the editable multi-page source-path.drawio for flow arrows. All unexpanded callees prevent any completeness claim.